SILICONLITE

Design and Verification of a Five-Stage Pipelined RISC Processor

Project domain	Digital Design / RTL Design and Verification
Implementation language	Verilog HDL (Verilog-2001 style)
Simulation tool	ModelSim Intel FPGA Edition 10.5b
Synthesis tool	Yosys 0.70+221
Author	Add student name / roll number
Institution	Add institution / department
Date	October 2026

Project report prepared from the supplied SiliconLite source tree and the recorded ModelSim and Yosys logs. Replace the author and institution placeholders before submission.

Abstract

SiliconLite is an educational five-stage pipelined processor implemented in Verilog. Its datapath is organized into Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages separated by pipeline registers. The implemented instruction subset includes selected register-register ALU operations, ADDI, word loads and stores, and conditional BEQ/BNE branches. The design includes operand forwarding, a load-use interlock, and branch redirection with flushing of younger instructions.

The submitted ModelSim transcript records successful compilation with zero errors and zero warnings, 12 directed register/memory checks passing, one load-use stall, two taken branches, and a final **RESULT: PASS**. The supplied Yosys log records successful generic synthesis, zero reported design problems at the check stage, and 112 generic logic cells after optimization. These generic cell counts are not physical area, power, or timing measurements.

Report structure

- Project objectives and scope
- Architecture and pipeline organization
- Instruction subset and hazard handling
- Verification methodology and observed results
- Synthesis results and limitations
- Reproducibility, future improvements, and conclusion

1. Introduction

Pipelining improves instruction throughput by allowing different instructions to occupy different processing stages during the same clock cycle. Instead of completing all instruction processing serially in one long datapath, a pipelined processor divides the work into stages and stores intermediate values in pipeline registers. This improves the educational visibility of datapath control, dependencies, and control-flow hazards, while introducing additional correctness requirements.

1.1 Objectives

- Implement a compact processor RTL with five pipeline stages.
- Support a defined educational subset of RV32I-style instruction encodings.
- Demonstrate forwarding for recently produced operands.
- Detect a load-use dependency and insert a pipeline bubble.
- Redirect the program counter and flush younger instructions for taken BEQ/BNE branches.
- Verify expected architectural register and data-memory values using a self-checking testbench.
- Run generic synthesis and record reproducible tool outputs.

1.2 Scope and exclusions

This is not a complete RISC-V RV32I implementation. The documented scope excludes byte and halfword loads/stores, shifts, unsigned comparisons, JAL/JALR, exceptions, CSRs, interrupts, and misaligned-access traps. Instruction and data memories are modeled as arrays for simulation; a target FPGA/ASIC implementation would require an appropriate memory mapping and implementation flow.

2. Architecture and Datapath

The design uses four sets of pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB. These registers carry instruction fields, operands, immediate values, destination-register identifiers, ALU results, memory data, and control signals between stages.

Stage	Primary responsibility	Pipeline boundary
IF — Fetch	Read instruction memory using the program counter and advance the PC.	IF/ID
ID — Decode	Decode opcode and fields, read register operands, generate immediate/control values, and detect load-use dependencies.	ID/EX
EX — Execute	Perform ALU operations, forward operands, compare branch operands, and calculate branch target.	EX/MEM
MEM — Memory	Perform word stores and obtain load data from the modeled data memory.	MEM/WB
WB — Write Back	Select ALU or load data and write the destination register.	Register file

2.1 Register file and memory model

The RTL contains 32 general-purpose 32-bit registers and sets register x0 to zero on active clock edges. Instruction and data memories are modeled as 256-entry, 32-bit arrays. Data-memory word addressing uses address bits [9:2], so the modeled accesses are word-aligned. The testbench initializes unused instruction memory entries with the ADDI x0,x0,0 encoding used as a NOP.

2.2 ALU operations and instruction classes

Class	Supported operations	Role
R-type	ADD, SUB, AND, OR, XOR, SLT	Register-register ALU
I-type	ADDI	Add sign-extended immediate
Load/store	LW, SW	Word memory access
Branch	BEQ, BNE	Conditional PC redirection

3. Pipeline Hazards and Control

3.1 Data forwarding

A consumer instruction may require a result that has not yet been written back to the register file. SiliconLite checks the EX/MEM and MEM/WB pipeline stages for matching destination and source register identifiers. When an eligible EX/MEM ALU result matches, it is forwarded; otherwise, a matching MEM/WB write-back value can be forwarded. The EX/MEM load value is not forwarded directly as an ALU result because load data is produced by the memory stage.

3.2 Load-use stall

When the instruction in ID/EX is a load and its destination matches a source operand of the instruction in IF/ID, the interlock holds the program counter and IF/ID register for one cycle and inserts a bubble into ID/EX. This allows the load to advance so that its result can be used safely. The recorded testbench observed one load-use stall.

3.3 Branch redirection and flushing

BEQ and BNE comparisons are resolved in the execute stage. When a branch is taken, the PC is redirected to the branch target and younger instructions are flushed by replacing their pipeline contents with NOP/bubble values. The directed test checks that wrong-path instructions do not overwrite their intended destination registers. Two taken branches were observed in the supplied run.

3.4 Control-flow caveat

This project is a learning implementation. It should not be represented as a complete production-ready CPU or a fully compliant RISC-V core. Further testing is required for all instruction combinations, branch outcomes, dependency distances, register x0 suppression, reset edge cases, and memory boundaries.

4. Verification Methodology

The testbench instantiates the CPU with explicit connections for all five top-level ports: clock, reset, debug PC, write-back value, and write-back write indicator. It generates a periodic clock, initializes the instruction/data memories, applies reset, executes a directed instruction sequence, and checks register and memory contents against expected values. It also records basic cycle, stall, branch, and instruction-class counters and writes a VCD waveform file.

4.1 Directed scenarios

- ADDI baseline values: x1 = 5 and x2 = 7.
- ALU dependency/forwarding: x3 = 12.
- Store/load sequence: memory word 0 = 12 and x4 = 12.
- Load-use dependency: x5 = 13 after loading x4.
- Branch compare and BEQ flush: x6 = 13, x7 = 42.
- BNE flush: x8 = 8.
- ALU decode checks: AND produces x9 = 5, OR produces x10 = 7, and SUB produces x11 = 2.

4.2 Verification evidence

Metric	Recorded result
Compilation	0 errors, 0 warnings
Register checks	11 checks passed
Memory check	memory[0] = 12 — PASS

Metric	Recorded result
Total directed checks	12 PASS checks
Active cycles	65
Load-use stalls	1
Taken branches	2
Instruction bins	R-type = 4, LW = 1, SW = 1, Branch = 2
Final status	**RESULT: PASS**

The values above are taken from the included build/simulation_final.log, which identifies ModelSim Intel FPGA Edition 10.5b and ends with RESULT: PASS, Errors: 0, Warnings: 0. Instruction-bin counters are lightweight observed-class counters, not a formal coverage percentage or complete ISA coverage model.

5. Generic Synthesis and QoR

The included Yosys script reads the RTL, checks the top-level hierarchy, lowers processes, performs optimization and memory handling, maps generic logic through techmap, reports statistics, and writes a generic Verilog netlist to build/siliconlite_generic_netlist.v.

5.1 Recorded synthesis statistics

Metric	Value from Yosys log
Synthesis tool	Yosys 0.70+221
Top module	siliconlite_cpu
Wires	336
Wire bits	8,983
Top-level ports	5 ports / 67 port bits
Generic logic cells	112
AND cells	49
NOT cells	1
Synchronous-reset DFF cells	33
XOR cells	29
Yosys check	0 design problems reported at check stage

Important interpretation: the 112-cell figure is a generic logic-cell count from Yosys, not a standard-cell area value. No target technology library, physical implementation, clock constraint, or static timing analysis result is included in the supplied evidence. Therefore, physical area, maximum frequency, worst slack, and power are not reported.

5.2 Artifacts

- build/yosys.log — detailed synthesis transcript and statistics.
- build/siliconlite_generic_netlist.v — generated generic Verilog netlist.
- scripts/synth_yosys.ys — synthesis command sequence.
- build/simulation_final.log — final ModelSim compilation and verification evidence.
- siliconlite.vcd and Waveforms/*.png — waveform data and screenshots.

6. Reproducibility

6.1 ModelSim simulation

From the SiliconLite project root, launch the configured ModelSim environment and run:

```
vsim -c -do scripts\run_modelsim.do
```

The run_modelsim.do script removes and recreates the work library, compiles the RTL and testbench, loads tb_siliconlite_cpu, runs all testbench activity, and exits. The final log should show zero compilation errors and warnings and RESULT: PASS.

6.2 Yosys synthesis

From a terminal where Yosys is available, run:

```
yosys -l build/yosys.log -s scripts/synth_yosys.ys
```

Confirm that the log completes and the generic netlist exists. Exact command syntax may need adjustment depending on the shell and installation. Preserve the actual generated log rather than manually typing statistics into a report.

7. Limitations and Future Improvements

The following improvements would make the design and its evidence stronger:

- Expand instruction support and add architectural reference-model checking for the implemented ISA subset.
- Add explicit tests for every ALU operation, both taken and not-taken branches, EX/MEM and MEM/WB forwarding, multiple dependency distances, and x0 write suppression.
- Add reset, memory-boundary, and long-running instruction-stream tests.
- Make the testbench report individual test identifiers, expected/actual values, and a machine-readable summary.
- Map the design to a specific FPGA or ASIC technology and document the memory implementation.
- Use a compatible target library, timing constraints, and static timing analysis before claiming physical area or timing.
- Add a public source repository URL and tag the exact revision used for the final simulation and synthesis logs.

8. Conclusion

SiliconLite demonstrates the core concepts of a five-stage pipelined processor: stage separation, ALU execution, load/store access, operand forwarding, load-use interlocking, and branch redirection with flushing. The supplied ModelSim transcript records a clean run with 12 passing directed checks, one load-use stall, two taken branches, and RESULT: PASS. The supplied Yosys transcript records successful generic synthesis with 112 generic logic cells and no reported problems at the design-check stage. These results support an educational RTL and functional-verification claim, while physical area, timing, power, and complete ISA compliance remain outside the measured scope.

Appendix A — Submission checklist

- Replace the author, roll number, department, and institution placeholders on the cover.
- Include the final RTL, testbench, scripts, verification log, Yosys log, generic netlist, and waveform screenshots.
- Generate the presentation from the same verified numbers and screenshots.
- Publish the source repository and place the real URL in LIVE_PROJECT_LINK.txt.
- Rebuild and inspect the final ZIP; do not include temporary tool libraries unless required.
- Ensure the report does not claim physical area, timing, power, or full RV32I compliance.

Evidence basis: supplied SiliconLite ZIP, README, verification plan, ModelSim final transcript, and Yosys synthesis log.